

Day 5 — JavaScript Fundamentals for Testers – II

Insurance Industry Hands-On Exercise Guide

Audience: Software testers and QA engineers learning JavaScript for UI/API automation.

Goal: Apply loops, functions, strings, arrays, objects, error handling, and debugging to realistic insurance test data.

Topic	Insurance testing application
Loops	Run multiple policy, customer, claim, or premium test cases
Functions	Create reusable validation utilities
Strings	Validate policy numbers, email addresses, claim descriptions
Arrays	Store multiple test values and expected results
Objects	Represent customer, policy, claim, and quote data
Error handling	Handle invalid or missing test data safely
Debugging	Identify common JavaScript mistakes in test scripts

Exercise 1 — Loop Through Insurance Applicant Ages

Use a **for loop** with ages 18, 25, 39, 67, 72. An applicant is eligible when age is between 18 and 65 inclusive. Print the age and Eligible/Not Eligible.

Reference solution:

```
let ages = [18, 25, 39, 67, 72];

for (let i = 0; i < ages.length; i++) {
  if (ages[i] >= 18 && ages[i] <= 65) {
    console.log(ages[i] + " - Eligible");
  } else {
    console.log(ages[i] + " - Not Eligible");
  }
}
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 2 — while Loop for Quote Attempts

A quote application allows up to 3 attempts to enter a valid premium. Process 0, -500, 12500 using a **while loop**. A premium greater than 0 is valid. Stop when a valid premium is found or after 3 attempts.

Reference solution:

```
let premiums = [0, -500, 12500];
let attempt = 0;

while (attempt < premiums.length) {
  console.log("Attempt " + (attempt + 1));
  if (premiums[attempt] > 0) {
    console.log("Valid premium: " + premiums[attempt]);
    break;
  }
  console.log("Invalid premium");
  attempt++;
}
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 3 — Reusable Policy Number Validation

Create **validatePolicyNumber(policyNumber)**. Rule: policy number starts with POL- followed by exactly 6 digits. Test POL-123456, POL-ABC123, POL-12345, AUTO-123456.

Reference solution:

```
function validatePolicyNumber(policyNumber) {
  return /^POL-\d{6}$/.test(policyNumber);
}

console.log(validatePolicyNumber("POL-123456")); // true
console.log(validatePolicyNumber("POL-ABC123")); // false
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 4 — Function Parameters and Return Values

Create `calculatePremium(basePremium, taxRate)` that returns premium including tax. Test 1000 at 13% and 2500 at 8%.

Reference solution:

```
function calculatePremium(basePremium, taxRate) {  
  return basePremium + (basePremium * taxRate / 100);  
}  
  
console.log(calculatePremium(1000, 13));  
console.log(calculatePremium(2500, 8));
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 5 — Insurance String Operations

For **"Vehicle damaged in parking lot"**: find length, uppercase it, check whether it contains "damaged", replace "parking lot" with "garage", and display the first 10 characters.

Reference solution:

```
let description = "Vehicle damaged in parking lot";  
  
console.log(description.length);  
console.log(description.toUpperCase());  
console.log(description.includes("damaged"));  
console.log(description.replace("parking lot", "garage"));  
console.log(description.substring(0, 10));
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 6 — Store Multiple Insurance Products

Create an array containing Auto, Home, Life, Travel, Health. Print the first product, last product, total count, and every product using a loop.

Reference solution:

```
let products = ["Auto", "Home", "Life", "Travel", "Health"];  
  
console.log(products[0]);  
console.log(products[products.length - 1]);  
console.log(products.length);  
  
for (let product of products) {  
  console.log(product);  
}
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 7 — Iterate Through Insurance Test Data

Create an array of three objects containing `customerName`, `age`, and `expectedResult`. Use `Anu/25/Eligible`, `Ravi/70/Not Eligible`, `Meena/40/Eligible`. Loop through and print each test case.

Reference solution:

```
let testData = [
  { customerName: "Anu", age: 25, expectedResult: "Eligible" },
  { customerName: "Ravi", age: 70, expectedResult: "Not Eligible" },
  { customerName: "Meena", age: 40, expectedResult: "Eligible" }
];

for (let testCase of testData) {
  console.log(
    testCase.customerName + " | Age: " +
    testCase.age + " | Expected: " +
    testCase.expectedResult
  );
}
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 8 — Work with a Policy Object

Create a policy object containing `policyNumber`, `customerName`, `product`, `premium`, and `status`. Read the values and print a sentence describing the policy.

Reference solution:

```
let policy = {
  policyNumber: "POL-123456",
  customerName: "Anu",
  product: "Auto",
  premium: 12500,
  status: "Active"
};

console.log("Policy " + policy.policyNumber +
  " belongs to " + policy.customerName +
  ". Product: " + policy.product +
  ". Status: " + policy.status + ".");
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 9 — Basic Error Handling

Write a function that throws **"Policy number is mandatory"** when `policyNumber` is missing. Use **`try...catch`** and test both valid and invalid policy objects.

Reference solution:

```
function validatePolicy(policy) {
  if (!policy.policyNumber) {
    throw new Error("Policy number is mandatory");
  }
  return "Policy data is valid";
}

try {
```

```

console.log(validatePolicy({ policyNumber: "POL-123456" }));
console.log(validatePolicy({ customerName: "Anu" }));
} catch (error) {
console.log("Validation error: " + error.message);
}

```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 10 — Debug This Insurance Test Script

The script should print customers whose age is at least 18. Find and fix the errors: the loop uses the wrong boundary and the if statement has an accidental semicolon.

Reference solution:

```

let customers = ["Anu", "Ravi", "Meena"];
let ages = [25, 17, 40];

for (let i = 0; i < customers.length; i++) {
  if (ages[i] >= 18) {
    console.log(customers[i] + " is eligible");
  }
}

```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Exercise 11 — Mini Insurance Validation Challenge

Build a reusable validation program for a quote containing customerName, email, age, and policyNumber. Create separate functions for required fields, age, email, and policy number. Print PASS/FAIL.

Reference solution:

```

function isRequired(value) {
  return value !== undefined && value !== null && value !== "";
}

function isValidAge(age) {
  return age >= 18 && age <= 65;
}

function isValidEmail(email) {
  return /^[^@s@]+@[^@s@]+\.[^@s@]+$/.test(email);
}

function isValidPolicyNumber(policyNumber) {
  return /^POL-\d{6}$/.test(policyNumber);
}

let quote = {
  customerName: "Anu",
  email: "anu@example.com",
  age: 35,
  policyNumber: "POL-123456"
};

let pass =
isRequired(quote.customerName) &&

```

```
isValidAge(quote.age) &&  
isValidEmail(quote.email) &&  
isValidPolicyNumber(quote.policyNumber);  
  
console.log(pass ? "PASS" : "FAIL");
```

Tester challenge: Modify the test data to include at least two negative scenarios and verify the result.

Day 5 — Final Hands-On Assignment

Scenario: You are testing an insurance quote application. The application receives multiple customer records and validates eligibility before a quote can be generated.

Create an array containing at least 5 customer objects. Each object should include name, age, email, policyNumber, product, and expectedResult.

Your JavaScript program should:

- Loop through every customer record.
- Validate mandatory fields.
- Validate age.
- Validate email format.
- Validate policy number.
- Print a PASS/FAIL result for each customer.
- Use at least two reusable functions.
- Use an object for each customer.
- Use try...catch for at least one validation failure.
- Include at least two intentionally invalid test cases.

Expected tester-style output

```
Test Case 1: Anu — PASS
Test Case 2: Ravi — FAIL — Invalid age
Test Case 3: Meena — PASS
Test Case 4: Kumar — FAIL — Invalid email
Test Case 5: Priya — FAIL — Invalid policy number
```

Debugging Checklist

- Check the browser console for syntax errors.
- Use console.log() to inspect variable values.
- Verify array indexes start at 0.
- Check loop conditions for < versus <=.
- Check whether a function returns a value.
- Check that object property names match exactly.
- Check for accidental semicolons after if conditions.
- Test one small logical step at a time.